Where Does the Loss Come From? Disentangling Token Pruning and Quantization in Compressed Vision–Language Document Retrieval

T Rithik Krishna*, Amgovath Navanitha*, Badavath Akhila*, E. Sathiya Lakshmi†
Department of Emerging Technologies, Mahatma Gandhi Institute of Technology (Autonomous)
Affiliated to Jawaharlal Nehru Technological University Hyderabad, Gandipet, Hyderabad 500075, Telangana, India
*{23261a6753, 23261a6704, 23261a6709}@mgit.ac.in †sathiyalakshmi_et@mgit.ac.in

Abstract—Vision–language retrievers such as ColPali remove optical character recognition from the document search pipeline by embedding rendered page images directly, but they replace one dense vector per page with roughly a thousand patch vectors, inflating the index by two to three orders of magnitude. The standard remedy is to prune visual tokens and to quantize the survivors, and systems that do both report large end-to-end compression figures. Those figures do not say which of the two stages the retrieval quality was actually paid to. We build OptiVision RAG, a model-agnostic compression layer that applies pixel-space saliency pruning, embedding-space redundancy collapsing and one-bit quantization to any ColBERT-style visual retriever, and we run a controlled ablation in which the corpus is encoded once and every configuration replays over identical cached vectors. We run that ablation twice: on a 60-page generated corpus with a 256M-parameter encoder, and on four ViDoRe test splits (1,780 pages, 1,325 queries) with the reference ColPali-v1.3 encoder. The attribution does not transfer between them. Under the small encoder the two stages are sharply asymmetric: both pruning stages together give a 3.5× token reduction for 3.9% of nDCG@5, while one-bit quantization alone costs 12.1% without discarding a single token and drives Kendall τ to 0.585. Under the reference encoder on real benchmark pages that asymmetry disappears: one-bit quantization costs between nothing and 3.7%, and the full pipeline reaches 53–60× compression at 94.6–103.4% retention. What fails instead is the pruning premise. Real pages are dense, so both pruning stages together yield only 1.7–1.9× rather than 3.5×, and a token-budget sweep that was flat under the small encoder falls to 66.7% retention at a 10% budget. A third experiment holds the corpus fixed and swaps only the encoder, which separates the two: the encoder governs what the codec costs (12.1% to 1.6% on identical pages), while the corpus governs what pruning buys (4.2× to 1.9× under one encoder). We conclude that per-stage attribution is necessary, and that it is a property of the encoder and corpus rather than of the compression layer — so a compression ratio for multi-vector retrieval should be published together with its attribution and the setting the attribution was measured in.

Index Terms—document retrieval, vision–language models, late interaction, token pruning, binary quantization, index compression, ColPali

I. Introduction

Retrieval over scanned and born-digital documents has traditionally been staged: optical character recognition (OCR) extracts text, a layout parser recovers structure, a text encoder produces embeddings, and a vector database serves them. Each stage is a source of error, and the first is the worst — OCR degrades on scans, handwriting, stamps and dense tables, and the layout, figures and formatting that a human reader uses to locate information are discarded before the retriever ever sees them.

Vision–language retrievers avoid the pipeline entirely. ColPali [1] renders each page as an image, splits it into a grid of patches, and embeds every patch with a vision–language backbone, scoring a query against a page with the late-interaction MaxSim operator inherited from ColBERT [2]. No text is ever extracted, and the approach is state of the art on the ViDoRe benchmark.

The cost is the index. A page is no longer one vector but roughly a thousand patch vectors of 128 dimensions each. At float32 this is approximately 448 KB per page, so a one-million-page archive requires an index on the order of 448 GB, which must remain resident for search to be fast. The accuracy is affordable; the memory is not.

Two families of remedy exist and are usually applied together. Token reduction removes patch vectors before they are stored, on the observation that a document page is mostly blank paper; it descends from work on accelerating vision transformers, such as DynamicViT [6] and token merging [7]. Quantization shrinks each surviving vector, from product quantization [4] to the one-bit hashing used in dense passage retrieval [5]. Combining them multiplies the savings, and a system that prunes to a quarter of the tokens and stores the rest as one-bit codes can honestly advertise a compression ratio above 100×.

That headline number is also where the reporting usually stops. It says how small the index became and how much retrieval quality was lost in total, but not which stage the quality was spent on. The distinction is not academic: the two stages have entirely different cost structures and entirely different upgrade paths. If most of the loss belongs to pruning, the productive research direction is a better saliency model. If most of it belongs to the codec, pruning harder is nearly free and the effort belongs in the quantizer instead. A single end-to-end figure cannot distinguish these cases, and the two prescriptions are opposites.

This paper performs that attribution. Our contributions are:

1) A model-agnostic compression layer. OptiVision RAG applies pixel-space saliency pruning, embedding-space redundancy collapsing and configurable quantization to the patch embeddings of any ColBERT-style visual retriever, leaving the encoder unmodified. Pruning decisions are made from transparent pixel statistics rather than a learned model, so the behaviour is interpretable and the operating point is a dial rather than a retrained network.
2) A controlled per-stage ablation. The corpus is encoded once and every configuration replays over identical cached vectors under exact brute-force MaxSim, so a change in the metric can be caused only by the compression setting. Alongside absolute retrieval metrics we report Kendall τ against the uncompressed baseline's own ranking, which isolates whether compression changed the retriever's mind independently of whether the retriever was right.
3) The finding that the attribution belongs to the setting, not to the layer. Under a 256M encoder on generated pages the two stages are sharply asymmetric, and the one-bit codec accounts for essentially all of the loss. Repeating the identical ablation with the reference ColPali encoder on four ViDoRe splits reverses this: the codec becomes nearly free, and what fails is the saliency premise that a document page is mostly blank paper. We report both regimes, give the operating points that follow in each, and argue that a compression ratio for multi-vector retrieval should be published with a per-stage attribution and the encoder and corpus it was measured on — because the attribution, not merely its magnitude, is what changes.

II. Related Work

A. Late-interaction and visual retrieval

ColBERT [2] introduced late interaction: rather than compressing a document into one vector, it retains one vector per token and scores a query by summing, over query tokens, the maximum similarity against any document token. ColBERTv2 [3] reduced the resulting storage with residual compression over centroids. ColPali [1] carried the mechanism into the visual domain, treating image patches as the document tokens, and Document Screenshot Embedding [9] showed that page screenshots are a viable universal document representation. These systems establish the accuracy that motivates our work and the index size that motivates compressing it.

B. Token reduction

DynamicViT [6] learns to discard uninformative tokens during inference and token merging [7] combines similar ones, both aimed at throughput for classification backbones. The objective differs from ours in a way that matters: a classifier needs only its pooled representation to survive, whereas a late-interaction retriever needs the surviving vectors to preserve a ranking over pages. A token that never wins a MaxSim maximum is free to remove; a token that decides a close ordering is not, and classification accuracy cannot detect the difference.

C. Quantization for retrieval

Product quantization [4] and binary hashing [5] are standard for compressing dense embeddings, and modern vector databases expose both alongside approximate nearest-neighbour structures such as HNSW [8]; Qdrant [10] additionally supports multi-vector MaxSim comparators with binary quantization and rescoring. These techniques are typically evaluated on single-vector retrieval, where each document contributes one vector and quantization error affects one score. Under MaxSim the error enters through a maximum over hundreds of vectors per page, which is a different statistical regime, and it is applied uniformly with no notion of which tokens matter.

D. Positioning

The individual components we use are not novel, and we do not claim them. What is missing from the literature is a controlled attribution of quality loss between pruning and quantization when both are applied to the same multi-vector visual index. Systems report the product of the two savings; we report the ratio of the two costs.

III. Method

A. Overview

OptiVision RAG sits between an unmodified visual encoder and the vector index (Fig. 1). A page is rendered to an image and encoded into $n \approx 875$ patch vectors of dimension $d = 128$. A saliency map computed directly from the page pixels selects which grid cells survive; a redundancy pass in embedding space collapses near-duplicate survivors; a quantizer compresses what remains; and the result is stored as a multi-vector page entry scored by MaxSim. The encoder is never fine-tuned and never sees the compression, which is what makes the layer transferable across ColPali, ColQwen2 and ColSmol checkpoints.

B. Stage 1: pixel-space saliency pruning

The premise that a page is mostly paper can be tested on the pixels, before the model runs and at negligible cost. For each cell of the patch grid we compute two signals on the greyscale page. Ink density is the mean positive deviation below the paper level, where the paper level is estimated as the 95th percentile of intensity rather than a fixed white point, so that yellowed or photocopied scans are not read as ink everywhere. Edge energy is the mean absolute first difference in both axes, which responds to glyph strokes, rules, table lines and stamp boundaries. Both are normalised by a high percentile rather than the maximum, so a single dark speck — dust, a punch hole, a staple shadow — cannot squash the rest of the page toward zero. The saliency of cell (r, c) is page image → visual encoder (frozen)

↓ $n \approx 875$ patch vectors, $d = 128$, float32

Stage 1: spatial pruning
ink density + edge energy, computed on pixels

↓ $\sim$356 vectors

Stage 2: redundancy pruning
greedy cosine clustering, $\theta = 0.92$

↓ $\sim$247 vectors

Stage 3: quantization
one-bit sign codes, or int8

↓ $\sim 247 \times 16$ bytes

multi-vector index, asymmetric MaxSim

Fig. 1. The compression layer. Vector counts are the measured means over the evaluation corpus. Only the three middle stages are ours; the encoder and the index are used as published.

Fig. 2. Spatial pruning on a page from the evaluation corpus. Left: the rendered page. Centre: the pixel-space saliency map. Right: the retained patch mask, green for kept and red for discarded — 233 of 768 grid patches (30%) survive, and after redundancy collapsing 233 of 875 vectors (27%) are stored. The retained region tracks the text block, and the margins, which occupy most of the page, cost nothing.

$$s_{rc} = \frac{w_{\text{ink}}\, \widehat{I}_{rc} + w_{\text{edge}}\, \widehat{E}_{rc}}{w_{\text{ink}} + w_{\text{edge}}}, \qquad (1)$$

with $w_{\text{ink}} = 0.6$ and $w_{\text{edge}} = 0.4$. Ink alone drops faint handwritten annotation; edges alone retain scanner noise on an empty margin; the weighted sum is robust to both and costs microseconds. Cells with $s_{rc} > \tau_{\text{blank}} = 0.02$ are kept, and the resulting mask is dilated by one cell in 4-connectivity, because glyphs rarely stop at a patch boundary and the ascender of a heading routinely falls into a neighbouring cell whose own density is below threshold. A floor of 8 cells guarantees that a nearly empty page remains retrievable. An alternative mode keeps the top k cells by saliency, which fixes the per-page budget — this is the mode used for the sweep in Section V-B, since a fixed budget is what index-size guarantees require. Fig. 2 shows the two modes' effect on a representative page.

C. Stage 2: embedding-space redundancy pruning

Stage 1 removes patches with no content. Stage 2 removes patches with duplicate content: the interior of a filled table cell, the middle of a thick rule, a run of uniform texture inside a photograph. These are salient to a pixel detector and yet contribute nothing to a MaxSim score. We visit the survivors in descending saliency order and greedily form clusters of vectors within cosine $\theta = 0.92$ of the representative, replacing each cluster with its renormalised mean.

This is close to free in ranking terms for a structural reason. MaxSim takes, per query vector, the maximum similarity over document vectors. If two document vectors lie within cosine θ of one another, the maximum over the pair and the similarity to their renormalised mean differ by an amount bounded by their angular spread, which is small by construction — so collapsing a tight cluster barely moves the score while removing one stored vector per duplicate. The argument does not extend to loose clusters, which is why θ is set high.

D. Stage 3: quantization and asymmetric scoring

A 128-dimensional float32 vector occupies 512 bytes; retaining only the sign of each dimension gives 128 bits, or 16 bytes — a flat 32× on everything the pruner allowed through. These vectors are L2-normalised with approximately zero-centred dimensions, so $\text{sign}(\mathbf{d})$ preserves the orthant of $\mathbf{d}$ and discards only its position within that orthant.

We score asymmetrically: the document side is binarised, the query side is not. A fully symmetric scheme reduces MaxSim to a popcount Hamming distance and is cheaper still, but it discards the query magnitudes — and those are precisely the part that can be afforded, since a query contributes on the order of twenty vectors against a corpus of millions. Asymmetric scoring costs one unpack and one float matrix product and recovers most of the ranking quality; the symmetric variant is retained only for comparison. For a query matrix Q and a page's code matrix C,

$$\text{MaxSim}(Q, C) = \sum_i \max_j \, \mathbf{q}_i \cdot \text{sign}(\mathbf{d}_j). \qquad (2)$$

We also evaluate an int8 codec at 4×. Because the components of an L2-normalised 128-dimensional vector concentrate near $1/\sqrt{d}$ — the observed mean absolute component is 0.071 and the observed maximum 0.363 — the textbook $\text{round}(127v)$ mapping, which assumes the data spans $[-1, 1]$, leaves roughly two thirds of the available levels permanently unused. Rescaling by the range the data actually occupies costs no additional storage and reduces the mean cosine error on real vectors from 3.28×10^{-4} to 8.20×10^{-5}.

E. Memory accounting

One implementation detail is load-bearing for the claims in this paper. A naive scorer decodes the packed index into a float32 matrix before computing similarities, which restores the full $d \times 4$ bytes per vector at query time and returns the entire saving — an index of 2.82 KB per page on disk demanded 90.12 KB per page in RAM to search. The storage claim was true while the system-level claim behind it was not. We therefore score in blocks sized against a memory budget, releasing each block before the next is allocated; peak resident memory then tracks the budget (measured at 0.93× of it) and is independent of corpus size, while producing scores bit-identical to the cached path. All index sizes reported below are computed from the pipeline's own byte accounting rather than from files on disk, so they are neither inflated by container overhead nor deflated by filesystem compression.

IV. Experimental Setup

A. Configuration

Three experiments share one pipeline, one measurement protocol and one code path, differing only in encoder and corpus. We label them E1, E2 and E3 throughout. E1 and E2 differ in both variables at once; E3 exists to separate them, and forms the fourth cell of a 2×2 over {small, reference} encoders and {generated, real} pages.

E1: small encoder, generated corpus. The encoder is vidore/colSmol-256M (ColIdefics3), float32, on CPU; hardware is a laptop with 8 GB of DDR4 and no GPU. The corpus is 60 generated pages (30 documents of 2 pages) rendered at A4, 150 dpi. The encoder emits 875 vectors per page before compression (768 page-grid patches, 64 thumbnail, 43 instruction).

E2: reference encoder, ViDoRe. The encoder is ColPali-v1.3 (3B parameters) in bfloat16 on a single Tesla T4. The corpus is four ViDoRe test splits — docvqa (500 pages, 451 queries), syntheticDocQA energy (500, 100), infovqa (500, 494) and tabfquad (280, 280) — for 1,780 pages and 1,325 queries in total, with the benchmark's own ground truth. The encoder emits 1,031 vectors per page.

E3: reference encoder, generated corpus. ColPali-v1.3 as in E2, over E1's corpus. The generator is deterministic, so regenerating at seed 7 reproduces the pages E1 measured rather than pages resembling them; we verified that the manifests match. The encoder emits 1,031 vectors per page, as in E2. E1 and E3 therefore share a corpus, E2 and E3 share an encoder, and each comparison isolates one variable.

Vector dimension is 128 in all three and every run uses seed 1337.

One implementation detail is worth recording, because it silently produces plausible numbers. ColPali is published as a LoRA adapter over a PaliGemma base. On current transformers and PEFT releases the adapter's key prefixes do not match the names of the injected modules, so the projection head is created and then left randomly initialised; the loader reports it as a missing key rather than as an error. A benchmark run in that state completes normally and yields a complete, well-formed, meaningless table. We therefore load the merged-weight checkpoint, which contains no adapter to mis-key, and our loader refuses to run if any parameter remains unmaterialised after loading.

B. Queries and ground truth

E1 has 72 queries in two deliberately different families. Sixty precise queries pair a subject with a unique identifier and have exactly one relevant page; these check that compression has not broken retrieval outright. Twelve topical queries name only a subject shared by several pages, so the metric depends on ordering a group of near-identical candidates — which is where compression damage becomes visible. Ground truth is exact by construction.

E2 uses the queries and relevance judgements shipped with each ViDoRe split, so the query distribution is the benchmark's rather than ours. The precise/topical distinction is a property of our generator and does not carry over, so E2 is reported per split instead.

C. Controls

Two choices make the ablation a controlled experiment rather than a leaderboard.

Exact scoring. Every quality figure comes from exhaustive brute-force MaxSim, not from an approximate index. An ANN structure would contribute its own recall loss and confound the attribution we are trying to make. The Qdrant backend is tested separately and is the deployment path, not the measurement path.

One encoding pass. The corpus is encoded once into a cache and every ablation row replays over those identical vectors. Nothing but the compression setting differs between rows, so a difference in a metric can have no other cause. Re-running the benchmark reproduces the reported figures exactly; the pipeline is deterministic given a fixed corpus and encoder.

D. Metrics

We report nDCG@5 with binary relevance (the standard ViDoRe metric), Recall@1 and Hit@5, index bytes per page, and the compression ratio against the uncompressed float32 index. We additionally report retention — nDCG@5 as a fraction of the baseline row — and Kendall τ-b against the baseline's own ranking. The last is the load-bearing metric here. Absolute quality answers "is this retriever any good?", which depends mostly on the choice of encoder; τ answers "did compression change the retriever's mind?", which is the only question this layer controls.

V. Results

Table I is the full E1 ablation and Fig. 3 plots it. Table II is the same ablation under E2 on the split with the most queries, and Table III gives retention across all four. Three findings follow. Each is stated as E1 supports it and then tested against E2, and only the first survives in the form E1 suggested.

TABLE I

E1. Ablation over the compression pipeline. 60 pages, 72 queries, ColSmol-256M, exact MaxSim. Rows share one encoding pass, so every difference is attributable to the compression setting alone. Retain is nDCG@5 relative to the baseline row; τ is Kendall τ-b against the baseline's own ranking.

Variant	What it isolates	Tok/pg	KB/pg	Compr.	nDCG@5	R@1	Hit@5	Retain	τ
baseline-float32	reference: every patch, full precision	875.0	448.00	1.0×	0.7823	0.5694	0.9722	100.0%	1.000
Pruning alone (no quantization)									
spatial-only	blank-patch pruning	356.1	182.32	2.5×	0.7602	0.5694	0.9444	97.2%	0.935
spatial+redundancy	both pruning stages	246.8	126.34	3.5×	0.7519	0.5139	0.9722	96.1%	0.866
Quantization alone (no pruning)									
int8-only	scalar quantization	875.0	112.00	4.0×	0.7787	0.5556	0.9722	99.5%	0.973
binary-only	one-bit quantization	875.0	14.00	32.0×	0.6875	0.4167	0.9444	87.9%	0.585
Both stages									
prune+int8	quality-first operating point	246.8	31.59	14.2×	0.7596	0.5278	0.9722	97.1%	0.866
optivision	size-first operating point	246.8	3.95	113.5×	0.6782	0.4028	0.9583	86.7%	0.606
optivision-aggressive	fixed 25% token budget	186.3	2.98	150.3×	0.6680	0.3611	0.9722	85.4%	0.602
Token-budget sweep, all with one-bit codes									
keep-50%	top 50% by saliency	288.1	4.61	97.2×	0.6704	0.4167	0.9444	85.7%	0.548
keep-40%	top 40% by saliency	263.6	4.22	106.2×	0.6721	0.4167	0.9583	85.9%	0.561
keep-30%	top 30% by saliency	241.6	3.87	115.9×	0.6721	0.4028	0.9583	85.9%	0.588
keep-20%	top 20% by saliency	206.1	3.30	135.9×	0.6676	0.3750	0.9722	85.3%	0.586
keep-10%	top 10% by saliency	162.9	2.61	171.8×	0.6700	0.4167	0.9306	85.6%	0.551

TABLE II

E2. The same ablation under ColPali-v1.3 on the ViDoRe docvqa split (500 pages, 451 queries), exact MaxSim, one encoding pass shared by every row. Compare with Table I: the binary rows have moved from the mid-80s to the mid-90s, and the pruning rows have lost most of their compression.

Variant	What it isolates	Tok/pg	KB/pg	Compr.	nDCG@5	R@1	Hit@5	Retain	τ
baseline-float32	reference: every patch, full precision	1031.0	527.87	1.0×	0.5841	0.4945	0.6608	100.0%	1.000
Pruning alone (no quantization)									
spatial-only	blank-patch pruning	846.3	433.30	1.2×	0.5794	0.4945	0.6519	99.2%	0.873
spatial+redundancy	both pruning stages	557.5	285.43	1.8×	0.5691	0.4856	0.6386	97.4%	0.715
Quantization alone (no pruning)									
int8-only	scalar quantization	1031.0	131.97	4.0×	0.5838	0.4989	0.6563	99.9%	0.919
binary-only	one-bit quantization	1031.0	16.50	32.0×	0.5625	0.4789	0.6364	96.3%	0.527
Both stages									
prune+int8	quality-first operating point	557.5	71.36	7.4×	0.5690	0.4878	0.6386	97.4%	0.712
optivision	size-first operating point	557.5	8.92	59.2×	0.5523	0.4545	0.6341	94.6%	0.510
optivision-aggressive	fixed 25% token budget	156.1	2.50	211.3×	0.4874	0.4080	0.5610	83.4%	0.409
Token-budget sweep, all with one-bit codes									
keep-50%	top 50% by saliency	373.9	5.98	88.2×	0.5404	0.4612	0.6142	92.5%	0.486
keep-40%	top 40% by saliency	309.0	4.94	106.8×	0.5276	0.4501	0.5987	90.3%	0.459
keep-30%	top 30% by saliency	239.8	3.84	137.6×	0.5076	0.4146	0.5942	86.9%	0.455
keep-20%	top 20% by saliency	165.3	2.64	199.6×	0.4778	0.3925	0.5543	81.8%	0.398
keep-10%	top 10% by saliency	86.0	1.38	383.6×	0.3899	0.3126	0.4634	66.7%	0.393

A. Finding 1: pruning is close to free, and buys less than it appears

Under E1, removing blank patches costs 2.8% of nDCG@5 for a 2.5× token reduction, and adding redundancy collapsing brings this to 3.5× for 3.9%. Rank agreement with the uncompressed baseline remains high throughout ($\tau = 0.87$–0.94), and spatial-only matches the baseline exactly on Recall@1 (0.5694): discarding 59% of the vectors does not cost a single top-1 hit.

Under E2 the quality claim holds more strongly and matters much less. Both pruning stages together retain 97.4–101.8% of nDCG@5 across the four splits, but the token reduction collapses from 3.5× to 1.7–1.9×. The reason is visible in the extreme case: on infovqa, spatial pruning keeps 1,000 of 1,031 patches and saves essentially nothing. Wide margins are a property of the documents our generator renders, not of documents in general, and ViDoRe — scans, infographics, dense tables — does not have them. The claim survives; the number behind it does not.

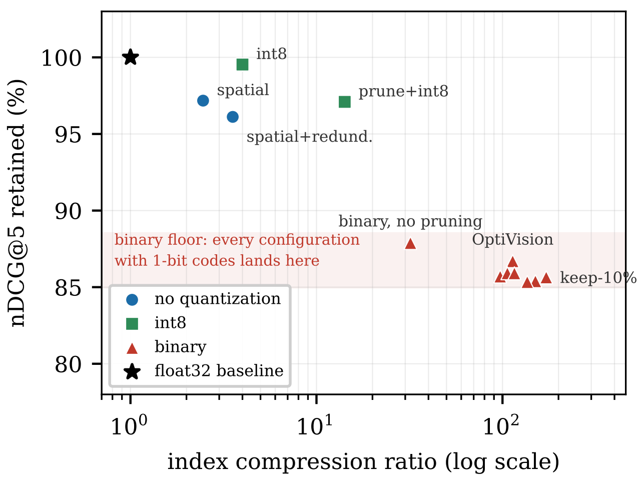

Fig. 3. E1. Compression ratio against nDCG@5 retention, by quantizer. The points separate by codec, not by token count: everything with one-bit codes falls into the shaded band between 85% and 88% retention whether it prunes nothing or prunes to a tenth, while the unquantized and int8 configurations stay above 96%. Fig. 4 shows what happens to this picture under E2.

TABLE III
E2. nDCG@5 retained across all four ViDoRe splits. The energy split has only 100 queries and reads above 100% for most rows; those differences are noise, not improvement. Compression ratios are rounded to whole numbers above 50×.

Variant	docvqa	energy	infovqa	tabfquad	Compr.
spatial-only	99.2%	101.1%	99.8%	99.8%	1.0–1.4×
spatial+redundancy	97.4%	101.8%	99.7%	99.2%	1.7–1.9×
int8-only	99.9%	101.6%	100.1%	99.9%	4.0×
binary-only	96.3%	100.6%	97.4%	100.3%	32.0×
prune+int8	97.4%	101.6%	99.5%	99.8%	6.7–7.5×
optivision	94.6%	103.4%	97.2%	95.6%	53–60×
keep-50%	92.5%	101.8%	94.7%	97.4%	81–91×
keep-10%	66.7%	89.8%	85.4%	81.6%	342–387×

B. Finding 2: the token budget is free only when the corpus is sparse

Under E1 the token-budget sweep is almost flat. Keeping the top 10% of patches — 163 vectors per page — retains 85.6% of nDCG@5, while keeping the top 50% — 288 vectors, a 1.8× larger index — retains 85.7%. The 0.1 point separating them is well inside the noise of 72 queries, which invites the conclusion that the token budget is not the operative variable once a codec is fixed.

Under E2 that conclusion fails outright. Keeping the top 10% retains 66.7% on docvqa, 81.6% on tabfquad and 85.4% on infovqa, against 92.5%, 97.4% and 94.7% at the top 50% (Table III). The flat sweep was an artifact of a corpus whose salient content fits comfortably inside the top 10% of patches; on pages where it does not, the aggressive setting is a genuine trade rather than a free 1.8×.

C. Finding 3: the binary penalty belongs to the encoder, not the codec

Under E1, binary-only loses 12.1% of nDCG@5 without discarding a single token, and its τ of 0.585 says it substantially reorders results rather than merely rescaling scores. Against this, int8-only costs 0.5% for 4× with $\tau = 0.973$. Every E1 configuration that includes one-bit codes lands in a narrow 85–88% retention band regardless of how much pruning is applied. On that evidence the codec is the binding constraint.

Under E2 the band is gone. binary-only retains 96.3% on docvqa and 97.4% on infovqa, and is indistinguishable from the baseline on the two splits with fewer queries. The full pipeline reaches 53–60× at 94.6–103.4% (Fig. 4). A 12.1% penalty became at most 3.7% under a change of encoder and corpus, with the codec itself untouched.

E1 and E2 differ in encoder and corpus together, so neither can say which variable moved. E3 holds the corpus fixed — ColPali over E1's own pages — and the two quantities separate cleanly (Table IV). With the corpus fixed, swapping the encoder takes the codec's cost from 12.1 to 1.6 points. With the encoder fixed, swapping the corpus takes pruning's yield from 4.2× to 1.9×. The encoder governs what the codec costs; the corpus governs what pruning buys. What E2 reported as one reversal is two effects with two causes.

E3 also rules out the explanation that suggests itself first, that a stronger retriever separates the gold page from its neighbours by a wider margin and so absorbs the same perturbation. On these identical pages ColPali is the weaker retriever — baseline nDCG@5 0.6954 against ColSmol's 0.7823, R@1 0.375 against 0.569 — and loses nothing at all to one-bit codes, with R@1 unchanged at 0.375 where ColSmol falls from 0.569 to 0.417. An encoder with less margin is the more robust one, so margin is not the mechanism. The τ column agrees: 0.762 for E3 against 0.585 for E1, at the same cutoff on the same corpus. Under the reference encoder the distortion is smaller, not better tolerated — a property of how much of a patch vector survives sign-thresholding, which we do not measure here and flag in Section VI-C.

Read together, the three findings do not support the statement that the quantizer is the binding constraint — that statement is an artifact of E1. At reference scale neither stage binds: compression to roughly 57× is nearly free, and the limit on the pipeline is how little a pixel-space saliency detector can remove from a dense page.

D. Two operating points

Table V summarises the practical consequence, and it differs between the two regimes. Under E1 the choice is a real exchange: prune + binary buys an 8× smaller index than prune + int8 and pays 10.4 points of retention for it, and which side is correct is a deployment question.

Under E2 that exchange collapses. Prune + binary gives roughly 8× the compression of prune + int8 at

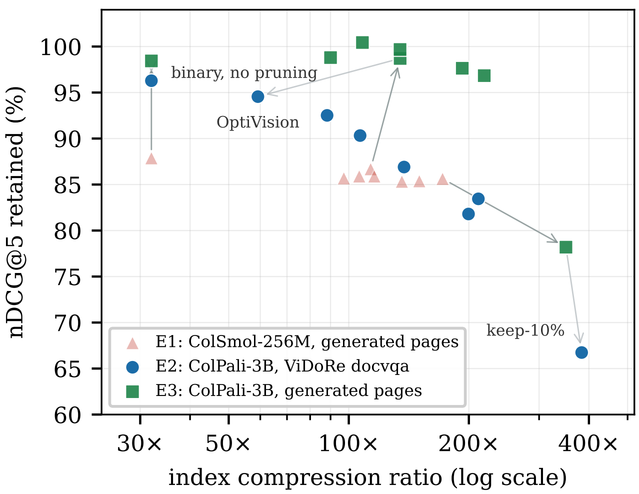

Fig. 4. The dissociation. Every configuration using one-bit codes, under all three experiments, at the same compression ratios. Under E1 (red) they sit in a band below 88% retention; under E2 (blue) the same configurations retain 94–97%. E3 (green) shares E1's pages and E2's encoder, and sits at or above E2, clear of E1's band — the codec's cost follows the encoder, not the corpus. Dark arrows join identical configurations from E1 to E3, which changes only the encoder; light arrows continue to E2, which changes only the corpus.

TABLE IV
Each quantity against its own control. E3 shares E1's corpus and E2's encoder, so every row changes exactly one thing. The codec's cost answers to the encoder; pruning's yield answers to the corpus. E2 figures are the docvqa split.

Quantity	Control	Change
one-bit codec cost	corpus fixed, encoder swapped	12.1 → 1.6 pts
one-bit codec cost	encoder fixed, corpus swapped	1.6 → 3.7 pts
pruning yield	corpus fixed, encoder swapped	3.6 → 4.2×
pruning yield	encoder fixed, corpus swapped	4.2 → 1.9×

the same nDCG@5, so on the headline metric the size-first configuration simply dominates. What still separates them is ordering: Kendall τ is 0.71–0.90 for prune + int8 against 0.51–0.70 for prune + binary. A pipeline whose output is a shortlist for a downstream reader should take the binary configuration and the 57×; one that shows a ranked list directly to a person should still take int8, and the justification is τ rather than nDCG@5. This is a narrower and more specific recommendation than E1 alone would have produced, and it is the opposite of the one E1 alone would have produced.

E3 makes the same recommendation more emphatically — 134.5× at 98.7% retention — but it should not be read as the headline. It combines the encoder that tolerates one-bit codes with the corpus that has the most blank paper to discard, which is the best case rather than a representative one. Its role here is to identify which variable supplies which part of the gain.

TABLE V
The configurations that follow from each experiment. Under E1 the two are a genuine exchange; under E2 and E3 the size-first point dominates on nDCG@5, and only τ separates them. E3 is the most favourable point we measure, but on sparse generated pages rather than real ones.

	Goal	Configuration	Compr.	Retain
E1	smallest index	prune + binary	113.5×	86.7%
E1	quality per byte	prune + int8	14.2×	97.1%
E2	smallest index	prune + binary	53–60×	94.6–103.4%
E2	quality per byte	prune + int8	6.7–7.5×	97.4–101.6%
E3	smallest index	prune + binary	134.5×	98.7%
E3	quality per byte	prune + int8	16.8×	102.3%

E. Latency

Query latency falls with the token count, since MaxSim cost is linear in stored vectors: under E1 from 3.25 ms at the float32 baseline to 0.70 ms at keep-10% on 60 pages, and under E2 on the 500-page docvqa split from 55.9 ms to 23.5 ms for the full pipeline and 4.4 ms at keep-10%. We do not emphasise these figures: the exact index used for measurement is not the structure a deployment would use, and query encoding dominates search entirely at this scale — 65.7 ms under E1 and 72.8 ms under E2.

VI. Discussion

A. Why the binary floor exists

For random unit vectors the expected cosine between $\mathbf{d}$ and $\mathrm{sign}(\mathbf{d})/\sqrt{d}$ is $\sqrt{2/\pi} \approx 0.798$, and the distortion is nearly the same for every document vector. Uniform distortion shifts scores far more than it reorders them, which is the usual argument for why binary quantization is safe. Our τ of 0.585 shows the argument is insufficient under MaxSim, and the reason is the maximum. A page score is a sum of per-query-vector maxima over hundreds of candidates; the identity of the $\arg\max$ is decided by small margins between the top few document vectors, and one-bit rounding perturbs exactly those margins. Distortion that is uniform in expectation is not uniform in its effect on an $\arg\max$, and this is why single-vector intuitions about binary quantization transfer poorly to late-interaction retrieval.

E3 tells us where this floor comes from. The $\arg\max$ argument above predicts damage whenever competing document vectors are close; it does not predict how close they are, which is a property of the representation. Under ColPali on the same pages, τ is 0.762 rather than 0.585 and R@1 is untouched, so sign-thresholding perturbs that encoder's $\arg\max$ far less. The floor is therefore not a fixed property of one-bit codes but of the encoder's patch geometry — how much of each vector's norm sits far enough from zero to survive the sign. This is precisely why an end-to-end retention figure cannot be attributed without naming the encoder it was measured on, and it is a directly testable prediction we leave to future work.

B. Implications

Three follow. First, and most strongly, a reported compression ratio for multi-vector retrieval should carry both a per-stage attribution and the encoder and corpus it was measured on. Our first two experiments differ only in those two variables and produce opposite attributions from the same code; a paper reporting either one alone would have supported a confident and wrong generalisation, and it took a third run to establish which variable was responsible for which half of the difference.

Second, the token budget is a free parameter exactly when the corpus is sparse, and not otherwise. E1 would have licensed pruning as aggressively as the detector allows; on ViDoRe that costs 25.8 points of retention between the top 50% and the top 10%. Sparsity is cheap to measure — it is the fraction of patches the saliency detector rejects — and should be reported alongside any token-reduction result.

Third, the research direction the attribution recommends is different in each regime, which is the practical point of making the attribution at all. Under a small encoder the codec binds and an intermediate two-bit or product-quantized document side would dominate both operating points. Under the reference encoder the codec is already nearly free, and the binding limit is the saliency detector: pixel statistics find little to remove on an infographic, which is where a learned or attention-derived selector would have to earn its cost. Shortlist rescoring remains attractive in both, since Hit@5 stays well above nDCG@5 throughout — the relevant page is usually retrieved and merely misordered.

C. Limitations

We state these plainly, as they bound the claims above.

Corpus scale. E2 uses 500-page subsets of each ViDoRe split rather than the full splits, so absolute nDCG@5 is not comparable with published leaderboard figures. Only the ratios between rows are the measurement here, and those are what we claim; but a larger corpus makes retrieval harder, and the compression penalties we report may be optimistic in absolute terms.

Statistical power. E1 has 72 queries and the energy split of E2 has 100, at which several rows read above 100% retention. Those readings are noise and we decline to interpret them; the docvqa (451) and infovqa (494) splits carry the E2 conclusions.

Two points, one family. We measure a 256M and a 3B encoder, both ColBERT-style vision–language retrievers. E3 separates encoder from corpus, but two points still do not locate a crossing: we cannot say at what scale the attribution changes, nor whether parameter count or embedding quality is the operative variable. We also do not measure the patch geometry that Section VI-A now rests on; that requires retaining the encode caches, which our runs discard.

Generator realism. On E3's pages ColPali-v1.3 scores below ColSmol-256M (nDCG@5 0.6954 against 0.7823). A 3B reference encoder losing to a 256M one is a fact about our generated pages rather than about the encoders, and it bounds what E1 and E3 can be taken to represent. It does not affect the comparisons we draw from them, which are within-encoder ratios.

What τ measures. Our τ is Kendall τ_b over the shared ids of two top-10 hit lists, not over the full ranking; the same E1 run scores 0.643 over all 60 pages, and the value moves with the cutoff (0.402 at top-5, 0.691 at top-20). It should therefore be read with its cutoff named, and τ values are comparable across experiments only when the candidate pool is a comparable fraction of the corpus — which holds for E1 against E3, both 60 pages, and not for either against E2's 500.

Scoring path. All quality numbers come from exact brute-force MaxSim, deliberately. A deployment on an approximate index would see additional recall loss from the ANN structure, which our figures do not include and were designed not to include.

VII. Conclusion

We built a model-agnostic compression layer for vision–language document retrieval and used it to ask which of its two stages the retrieval quality is actually paid to. Asking the question twice is what the paper turns on. Under a 256M encoder on generated pages the two stages are sharply asymmetric: pruning costs 3.9% of nDCG@5 for a 3.5× token reduction while one-bit quantization costs 12.1% having discarded no tokens at all, and every binary configuration lands in the same 85–88% band. Under ColPali-v1.3 on four ViDoRe splits the same code produces the opposite attribution: one-bit quantization costs at most 3.7%, the full pipeline reaches 53–60× at 94.6–103.4% retention, and what fails instead is the premise that a page is mostly blank paper — pruning yields 1.7–1.9× on real pages, and a token-budget sweep that was flat becomes a 25.8-point drop.

A third experiment holds the corpus fixed and swaps only the encoder, which separates the two quantities that reversal confounded: the encoder governs what the codec costs (12.1% to 1.6% on identical pages) and the corpus governs what pruning buys (4.2× to 1.9× under one encoder). It also rules out retrieval margin as the mechanism, since the reference encoder is the weaker retriever on those pages and still loses nothing to one-bit codes. The conclusion is therefore not that the codec binds, nor that the token count binds, but that neither is a property of the compression layer alone. A compression ratio for multi-vector retrieval should be published with a per-stage attribution and with the encoder and corpus that attribution was measured on, because the attribution itself is what moves.

Future work follows directly: measure the patch geometry that decides how much sign-thresholding costs an encoder, place a two-bit or product-quantized codec between the current operating points for the small-encoder regime, replace pixel-space saliency with a learned selector for the dense-page regime, and test shortlist rescoring, which our Hit@5 figures suggest should recover most of the remaining ranking loss at negligible cost.

Acknowledgment

The authors thank the Department of Emerging Technologies, Mahatma Gandhi Institute of Technology, for supporting this work.

References

[1] M. Faysse, H. Sibille, T. Wu, B. Omrani, G. Viaud, C. Hudelot and P. Colombo, "ColPali: Efficient Document Retrieval with Vision Language Models," in Proc. Int. Conf. on Learning Representations (ICLR), 2025.

[2] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in Proc. 43rd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval, 2020, pp. 39–48.

[3] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts and M. Zaharia, "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction," in Proc. NAACL-HLT, 2022, pp. 3715–3734.

[4] H. Jégou, M. Douze and C. Schmid, "Product Quantization for Nearest Neighbor Search," IEEE Trans. Pattern Anal. Mach. Intell., vol. 33, no. 1, pp. 117–128, 2011.

[5] I. Yamada, A. Asai and H. Hajishirzi, "Efficient Passage Retrieval with Hashing for Open-domain Question Answering," in Proc. ACL-IJCNLP, 2021, pp. 979–986.

[6] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou and C.-J. Hsieh, "DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 13937–13949.

[7] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer and J. Hoffman, "Token Merging: Your ViT But Faster," in Proc. Int. Conf. on Learning Representations (ICLR), 2023.

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, 2020.

[9] X. Ma, S.-C. Lin, M. Li, W. Chen and J. Lin, "Unifying Multimodal Retrieval via Document Screenshot Embedding," in Proc. EMNLP, 2024, pp. 6600–6611.

[10] Qdrant, "Multivectors, Binary Quantization and Rescoring," Qdrant Documentation. [Online]. Available: https://qdrant.tech/documentation/